

Quiz-03-Intro-Lexers

Scanner API

The scanner member function "token" returns a list of the tokens that can recognize

☐ True

☐ False

Programs for Lexical Analysis

Scanner (sometimes called lexer)

Defined by a list of tokens and definitions:

- ARTICLE
- NOUN
- VERB
- ADJECTIVE

Tokens

= {The, A, My, Your}
= {Dog, Car, Computer}
= {Ran, Crashed, Accelerated}
= {Purple, Spotted, Old}

Tokens Definitions

Original program:

Lex

[https://en.wikipedia.org/wiki/Lex_\(software\)](https://en.wikipedia.org/wiki/Lex_(software))

Popular implementations

Flex

Scanner API

```
# Constructor, generates a Scanner  
s = ScannerGenerator(tokens)  
  
# The string we want to do  
# lexical analysis on  
s.input("My Old Computer Crashed")  
  
# Returns the next lexeme  
s.token()
```

```
> s = ScannerGenerator(tokens)
> s.input("My Old Computer Crashed")
> s.token()
(ARTICLE, "My")
> s.token()
(ADJECTIVE, "Old")
> s.token()
(NOUN, "Computer")
> s.token()
(VERB, "Crashed")
> s.token()
None
```

Scanning vs. Parsing

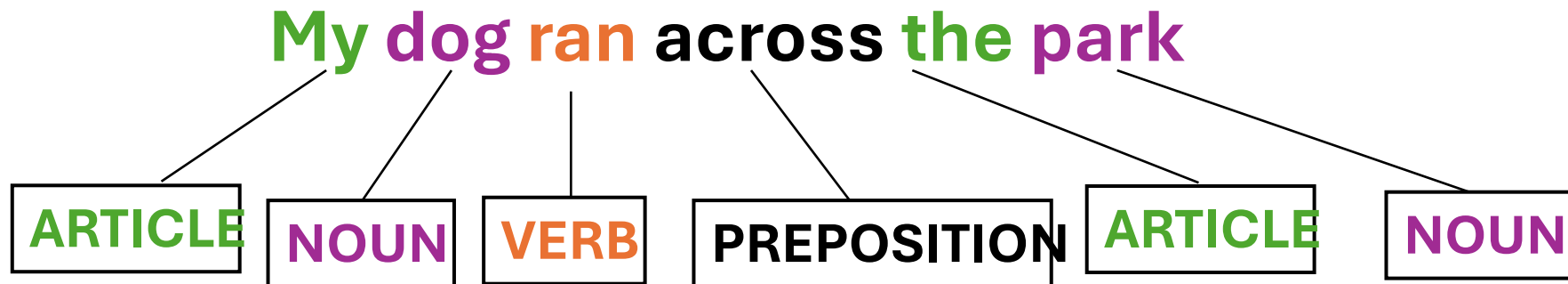
A scanner should make sure that the sequence of lexemes is valid, e.g. the scanner should make sure two numbers are separated by a valid operator.

☐ True

☐ False

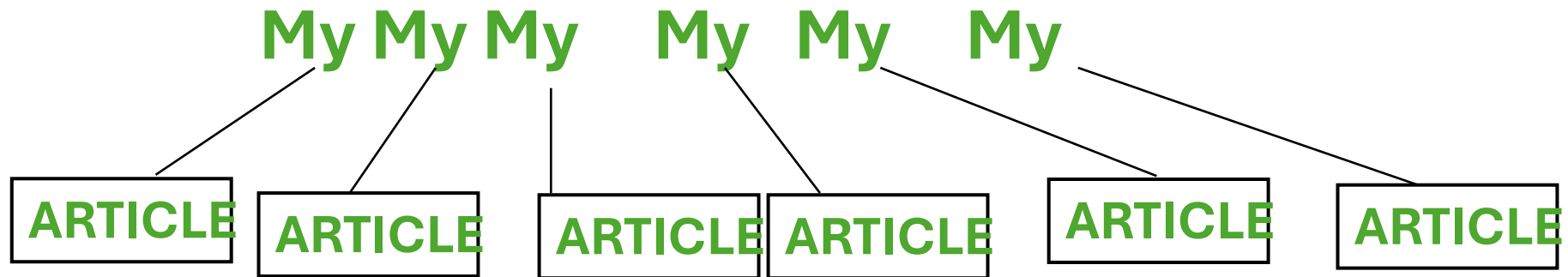
Parsing is the first step in a compiler

- How do we parse a sentence in English?



Parsing is the first step in a compiler

- How do we parse a sentence in English?



Lexical analysis doesn't care about the order of tokens. Just so long as there are valid tokens.

Programs for Lexical Analysis

Scanner (sometimes called lexer)

Defined by a list of tokens and definitions:

- ARTICLE
- NOUN
- VERB
- ADJECTIVE

Tokens

= {The, A, My, Your}
= {Dog, Car, Computer}
= {Ran, Crashed, Accelerated}
= {Purple, Spotted, Old}

Tokens Definitions

Original program:

Lex

[https://en.wikipedia.org/wiki/Lex_\(software\)](https://en.wikipedia.org/wiki/Lex_(software))

Popular implementations

Flex

Parsing is the first step in a compiler

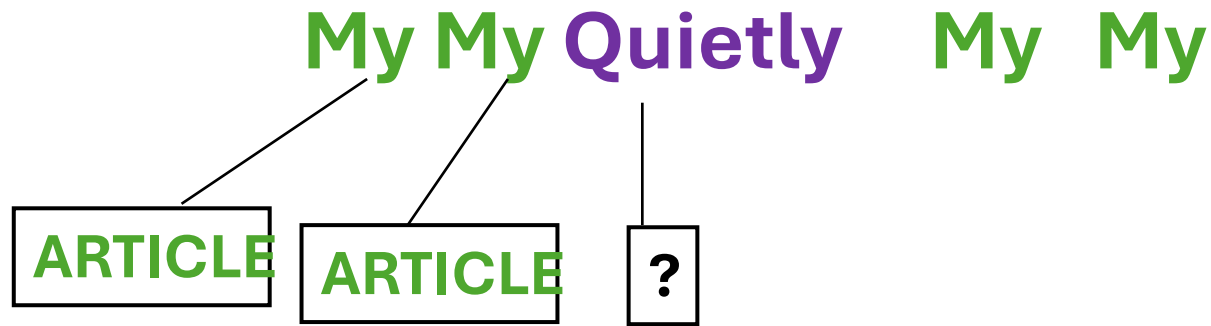
- How do we parse a sentence in English?

My My Quietly My My My

What happens here?

Parsing is the first step in a compiler

- How do we parse a sentence in English?



What happens here?

Scanner error here. Many scanners stop and report the error location

Parsing is the first step in a compiler

- How do we parse a sentence in English?



What happens here?

Scanner error here. Some scanners try to recover and keep going (difficult, and requires ad hoc rules)

Scanning vs. Parsing

A scanner should make sure that the sequence of lexemes is valid, e.g. the scanner should make sure two numbers are separated by a valid operator.

☐ True

☐ False

False! The order of tokens will be checked by the parser later on!

Scanning a simple PL statement

How many lexemes do you think the following statement should have?

```
for (int i = 0; i <=5; i++)
```

What lexemes do you think they should be?

Scanning a simple PL statement

```
for (int i = 0; i <= 5; i++)
```

Scanning a simple PL statement

```
for (int i = 0; i <= 5; i++)
```

```
[ (ID, "for"),      (PAR, "("), (ID, "int"), (ID, "i"),  
  (ASSIGN, "="),    (NUM, "0"),  (SEMI, ";"), (ID, "i"),  
  (LE, "<="),        (NUM, "5"),  (SEMI, ";"), (ID, "i"),  
  (INCR, "++"),     (PAR, ")") ]
```


Scanning a simple PL statement

```
for (int i = 0; i <= 5; i++)
```

```
[ (ID, "for"), (PAR, "("), (ID, "int"), (ID, "i"),  
  (ASSIGN, "="), (NUM, "0"), (SEMI, ";"), (ID, "i"),  
  (LE, "<="), (NUM, "5"), (SEMI, ";"), (ID, "i"),  
  (INCR, "++"), (PAR, ")") ]
```

Why not: "<" and "=" separately?

Scanning a simple PL statement

```
for (int i = 0; i <= 5; i++)
```

```
[ (ID, "for"), (PAR, "("), (ID, "int"), (ID, "i"),  
  (ASSIGN, "="), (NUM, "0"), (SEMI, ";"), (ID, "i"),  
  (LE, "<="), (NUM, "5"), (SEMI, ";"), (ID, "i"),  
  (INCR, "++"), (PAR, ")") ]
```

Should these be the same token?

Scanning a simple PL statement

```
for (int i = 0; i <= 5; i++)
```

```
[ (ID, "for"), (LPAR, "("), (ID, "int"), (ID, "i"),  
  (ASSIGN, "="), (NUM, "0"), (SEMI, ";"), (ID, "i"),  
  (LE, "<="), (NUM, "5"), (SEMI, ";"), (ID, "i"),  
  (INCR, "++"), (RPAR, ")") ]
```

Should these be the same token? Probably not